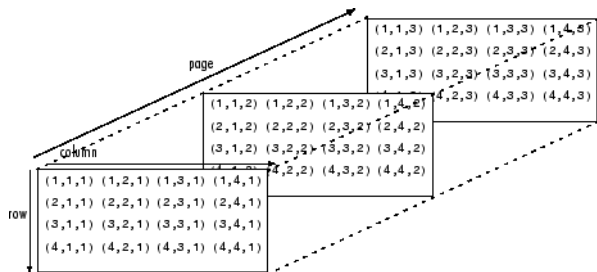


Arrays (vectors, matrices, lists)



Constructing a vector (1/3)

A vector is an array with 1 dimension

Arrays contain ordered collections

Definition:

Let a collection of n items: $item_1, item_2, \dots, item_n$

$[item_1, item_2, \dots, item_n]$

Julia is 1-based indexing

Examples:

```
julia> ["Austria", "France", "Belgium"]
```

```
julia> primeNumbers = [2, 3, 5, 7, 11]
```

```
julia> primeNumbers = [2, 3, 5, "seven", 11]
```

Constructing a vector (1/3)

A vector is an array with 1 dimension

Arrays contain ordered collections

Definition:

Let a collection of n items: $item_1, item_2, \dots, item_n$

$[item_1, item_2, \dots, item_n]$

Julia is 1-based indexing

Examples:

```
julia> ["Austria", "France", "Belgium"]
```

```
julia> primeNumbers = [2, 3, 5, 7, 11]
```

```
julia> primeNumbers = [2, 3, 5, "seven", 11]
```

Constructing a vector (2/3)

Typed but non initialized vector:

```
Array{type}(undef, size)
```

```
Vector{type}(undef, size)
```

Examples:

```
julia> v = Array{Int64}(undef, 5)
```

```
julia> v = Vector{Int64}(undef, 5)
```

Constructing a vector (2/3)

Typed but non initialized vector:

```
Array{type}(undef, size)
```

```
Vector{type}(undef, size)
```

Examples:

```
julia> v = Array{Int64}(undef, 5)
```

```
julia> v = Vector{Int64}(undef, 5)
```

Constructing a vector (3/3)

Typed and initialized vector:

```
zeros(type, size)
```

```
ones(type, size)
```

See also `collect` and `fill` functions

Examples:

```
julia> v0 = zeros{Int64,5}
```

```
julia> v1 = ones{Int64,5}
```

```
julia> v3to6 = collect(3:6)
```

```
julia> v10 = fill(10,5)
```



Constructing a vector (3/3)

Typed and initialized vector:

```
zeros(type, size)
```

```
ones(type, size)
```

See also `collect` and `fill` functions

Examples:

```
julia> v0 = zeros{Int64,5}
```

```
julia> v1 = ones{Int64,5}
```

```
julia> v3to6 = collect(3:6)
```

```
julia> v10 = fill(10,5)
```



Accessing and editing a vector (1/3)

Access to individual data inside an array by indexing into the array:

```
nameArray [index]
```

```
julia> primeNumbers[3]
```

Access to the last data inside an array:

```
nameArray [end]
```

```
julia> primeNumbers[end]
```

Access to multiple consecutive data inside an array:

```
nameArray [indexStart:indexEnd]
```

```
julia> primeNumbers[3:4]
```



Accessing and editing a vector (1/3)

Access to individual data inside an array by indexing into the array:

```
nameArray [index]
```

```
julia> primeNumbers[3]
```

Access to the last data inside an array:

```
nameArray [end]
```

```
julia> primeNumbers[end]
```

Access to multiple consecutive data inside an array:

```
nameArray [indexStart:indexEnd]
```

```
julia> primeNumbers[3:4]
```



Accessing and editing a vector (1/3)

Access to individual data inside an array by indexing into the array:

```
nameArray [index]
```

```
julia> primeNumbers[3]
```

Access to the last data inside an array:

```
nameArray [end]
```

```
julia> primeNumbers[end]
```

Access to multiple consecutive data inside an array:

```
nameArray [indexStart:indexEnd]
```

Accessing and editing a vector (1/3)

Access to individual data inside an array by indexing into the array:

```
nameArray [index]
```

```
julia> primeNumbers[3]
```

Access to the last data inside an array:

```
nameArray [end]
```

```
julia> primeNumbers[end]
```

Access to multiple consecutive data inside an array:

```
nameArray [indexStart:indexEnd]
```

```
julia> primeNumbers[3:4] 
```

Accessing and editing a vector (2/3)

Edit an existing element of an array (arrays are mutable):

```
nameArray[index] = item
```

```
julia> primeNumbers[2] = "three"
```

Edit multiple consecutive element of an array:

```
nameArray[indexStart:indexEnd] = [item1, ..., itemn]
```

```
julia> v[2:3] = [3, 7]
```

Accessing and editing a vector (2/3)

Edit an existing element of an array (arrays are mutable):

```
nameArray[index] = item
```

```
julia> primeNumbers[2] = "three"
```

Edit multiple consecutive element of an array:

```
nameArray[indexStart:indexEnd] = [item1, ..., itemn]
```

```
julia> v[2:3] = [3, 7]
```

Accessing and editing a vector (2/3)

Edit an existing element of an array (arrays are mutable):

```
nameArray [index] = item
```

```
julia> primeNumbers[2] = "three"
```

Edit multiple consecutive element of an array:

```
nameArray [indexStart:indexEnd] = [item1, ..., itemn]
```

```
julia> v[2:3] = [3, 7]
```

Accessing and editing a vector (3/3)

`push!` function adds an element to the end of an array

```
push!(nameArray, item)
```

```
julia> push!(primeNumbers, 13)
```

`pop!` function removes the last element of an array

```
pop!(nameArray)
```

```
julia> pop!(primeNumbers)
```

Accessing and editing a vector (3/3)

`push!` function adds an element to the end of an array

```
push!(nameArray, item)
```

```
julia> push!(primeNumbers, 13)
```

`pop!` function removes the last element of an array

```
pop!(nameArray)
```

```
julia> pop!(primeNumbers)
```


Accessing and editing a vector (3/3)

`push!` function adds an element to the end of an array

```
push!(nameArray, item)
```

```
julia> push!(primeNumbers, 13)
```

`pop!` function removes the last element of an array

```
pop!(nameArray)
```

```
julia> pop!(primeNumbers)
```

Constructing a matrix (1/4)

A matrix as multi-dimensional array

Definition:

Let a collection of items stored in a (e.g.) $n \times m$ grid (2D array),

$$[t_{1,1} \ t_{1,2} \dots t_{1,m} \ ; \ t_{2,1} \ t_{2,2} \dots t_{2,m} \ ; \ \dots \ ; \ t_{n,1} \ t_{n,2} \ \dots t_{n,m}]$$

Examples:

```
julia> [1 2 ; 3 4]
```

```
julia> T = ['a' 'b' 'c' ; 'x' 'y' 'z']
```

Constructing a matrix (1/4)

A matrix as multi-dimensional array

Definition:

Let a collection of items stored in a (e.g.) $n \times m$ grid (2D array),

$$[t_{1,1} \ t_{1,2} \dots t_{1,m} \ ; \ t_{2,1} \ t_{2,2} \dots t_{2,m} \ ; \ \dots \ ; \ t_{n,1} \ t_{n,2} \ \dots t_{n,m}]$$

Examples:

```
julia> [1 2 ; 3 4]
```

```
julia> T = ['a' 'b' 'c' ; 'x' 'y' 'z']
```

Constructing a matrix (1/4)

A matrix as multi-dimensional array

Definition:

Let a collection of items stored in a (e.g.) $n \times m$ grid (2D array),

$$[t_{1,1} \ t_{1,2} \dots t_{1,m} \ ; \ t_{2,1} \ t_{2,2} \dots t_{2,m} \ ; \ \dots \ ; \ t_{n,1} \ t_{n,2} \ \dots t_{n,m}]$$

Examples:

```
julia> [1 2 ; 3 4]
```

```
julia> T = ['a' 'b' 'c' ; 'x' 'y' 'z']
```

Constructing a matrix (2/4)

A matrix as an array of arrays

Definition:

Let a collection of items stored in (e.g.) a $n \times m$ grid (2D array),

```
[ [t11 t12...t1m], [t21 t22...t2m], [...], [t31 t32 ...t3m] ]
```

Examples:

```
julia> [ [1 2], [3 4] ]
```

```
julia> A = [ [1 2], [3 4 6], [1] ]
```

Constructing a matrix (2/4)

A matrix as an array of arrays

Definition:

Let a collection of items stored in (e.g.) a $n \times m$ grid (2D array),

```
[ [t11 t12...t1m], [t21 t22...t2m], [...], [t31 t32 ...t3m] ]
```

Examples:

```
julia> [ [1 2], [3 4] ]
```

```
julia> A = [ [1 2], [3 4 6], [1] ]
```

Constructing a matrix (2/4)

A matrix as an array of arrays

Definition:

Let a collection of items stored in (e.g.) a $n \times m$ grid (2D array),

```
[ [t11 t12...t1m], [t21 t22...t2m], [...], [t31 t32 ...t3m] ]
```

Examples:

```
julia> [ [1 2], [3 4] ]
```

```
julia> A = [ [1 2], [3 4 6], [1] ]
```

Constructing a matrix (3/4)

Typed but non initialized matrix:

```
Array{type}(undef, dim1, dim2, ..., dimn)
```

```
Matrix{type}(undef, dim1, dim2, ..., dimn)
```

Examples:

```
julia> m = Array{Int64}(undef, 2, 3)
```

```
julia> m = Matrix{Int64}(undef, 2, 3)
```


Constructing a matrix (3/4)

Typed but non initialized matrix:

```
Array{type}(undef, dim1, dim2, ..., dimn)
```

```
Matrix{type}(undef, dim1, dim2, ..., dimn)
```

Examples:

```
julia> m = Array{Int64}(undef, 2, 3)
```

```
julia> m = Matrix{Int64}(undef, 2, 3)
```

Constructing a matrix (4/4)

Typed and initialized matrix:

```
zeros(type, dim1, dim2, ..., dimn)
```

```
ones(type, dim1, dim2, ..., dimn)
```

See also `fill` function

Examples:

```
julia> m0 = zeros(Float64,2,3,4)
```

```
julia> m1 = ones(Float64,2,3,4)
```

```
julia> fill( $\pi$ ,2,3)
```

Constructing a matrix (4/4)

Typed and initialized matrix:

```
zeros(type, dim1, dim2, ..., dimn)
```

```
ones(type, dim1, dim2, ..., dimn)
```

See also `fill` function

Examples:

```
julia> m0 = zeros(Float64,2,3,4)
```

```
julia> m1 = ones(Float64,2,3,4)
```

```
julia> fill( $\pi$ ,2,3)
```

Accessing a matrix

General syntax for indexing into an n-dimensional array:

```
nameArray [index1, index2, ... , indexn]
```

```
julia> m[2,1]
```

General syntax for indexing an array of arrays:

```
nameArray [index1] [index2] ... [indexn]
```

```
julia> A[2][3]
```

All indices within an entire dimension or across the entire array:

```
:
```

```
julia> T[:,2]
```

Accessing a matrix

General syntax for indexing into an n-dimensional array:

```
nameArray [index1, index2, ... , indexn]
```

```
julia> m[2,1]
```

General syntax for indexing an array of arrays:

```
nameArray [index1] [index2] ... [indexn]
```

```
julia> A[2][3]
```

All indices within an entire dimension or across the entire array:

```
:
```

```
julia> T[:,2]
```

Accessing a matrix

General syntax for indexing into an n-dimensional array:

```
nameArray [index1, index2, ... , indexn]
```

```
julia> m[2,1]
```

General syntax for indexing an array of arrays:

```
nameArray [index1] [index2] ... [indexn]
```

```
julia> A[2][3]
```

All indices within an entire dimension or across the entire array:

```
:
```

```
julia> T[:,2]
```

Dimensions of an array

```
size(array, [dim])
```

```
julia> size(T)  
julia> size(T,2)
```

```
length(array)
```

```
julia> length(T)
```

Dimensions of an array

```
size(array, [dim])
```

```
julia> size(T)  
julia> size(T,2)
```

```
length(array)
```

```
julia> length(T)
```


Warning 1:

Observe this:

```
julia> [2, 3, 5, 7]  
julia> [2 3 5 7]
```

Return an array with the same data as *A*, but with different dimension sizes or number of dimensions

```
reshape(array, dims)
```

```
julia> reshape([2 3 5 7], 4)
```

```
julia> reshape([2 3 5 7], (2,2))
```

Warning 1:

Observe this:

```
julia> [2, 3, 5, 7]  
julia> [2 3 5 7]
```

Return an array with the same data as *A*, but with different dimension sizes or number of dimensions

```
reshape(array, dims)
```

```
julia> reshape([2 3 5 7], 4)
```

```
julia> reshape([2 3 5 7], (2,2))
```

Warning 2 (1/2)

Observe this:

```
julia> g = [2 3 4 5]
julia> h = g

julia> println(g)
julia> println(h)

julia> g[2] = 0

julia> println(h)
```

Warning 2 (2/2)

```
copy(array)
```

```
julia> copy(T)
```

```
deepcopy(array)
```

```
julia> deepcopy(T)
```

```
similar(array)
```

```
julia> similar(T)
```

Comprehension to construct arrays

```
[ fct(var1, var2, ...) for var1=val1, var2=val1, ...]
```

```
julia> [i for i in 1:5]
```

```
julia> [i+j for i in 1:2 for j in 1:4]
```

```
julia> [i for i in 1:2, j in 1:4]
```

```
julia> [i for i in 1:10 if i % 2 == 1]
```

```
julia> [exp(i) for i in 1:3]
```

Array and operations on a list (1/2)

```
julia> L=[1,2,3,4,5]
```

Adding elements

- ▶ at the end

```
julia> push!(L,10)
```

- ▶ at the front

```
pushfirst!(array,item)
```

```
julia> pushfirst!(L,0)
```

- ▶ (replacing) at the given index

```
splice!(array,position(s),item(s))
```

```
julia> splice!(L,3,[1 7 4])
```

Array and operations on a list (1/2)

```
julia> L=[1,2,3,4,5]
```

Adding elements

- ▶ at the end

```
julia> push!(L,10)
```

- ▶ at the front

```
pushfirst!(array,item)
```

```
julia> pushfirst!(L,0)
```

- ▶ (replacing) at the given index

```
splice!(array,position(s),item(s))
```

```
julia> splice!(L,3,[1,2] 7 4)
```

Array and operations on a list (2/2)

```
julia> L=[1,2,3,4,5]
```

Deleting elements

- ▶ at the end

```
julia> pop!(L)
```

- ▶ at the front

```
popfirst!(array, item)
```

```
julia> popfirst!(L)
```

- ▶ at the given index

```
splice!(array, position(s))
```

```
julia> splice!(L, 2)
```


Array and operations on a list (2/2)

```
julia> L=[1,2,3,4,5]
```

Deleting elements

- ▶ at the end

```
julia> pop!(L)
```

- ▶ at the front

```
popfirst!(array, item)
```

```
julia> popfirst!(L)
```

- ▶ at the given index

```
splice!(array, position(s))
```

```
julia> splice!(L,2)
```

